

Factory Design Pattern

Main logic is to separate our business logic from object creation. When we put both of them together then code become complex. So, this design pattern comes which says to keep object creation in different class.

Note:- In Strategy design pattern we assume that object is already created somewhere. we just need to use methods like:-

Robot * r = new Robot (_____)

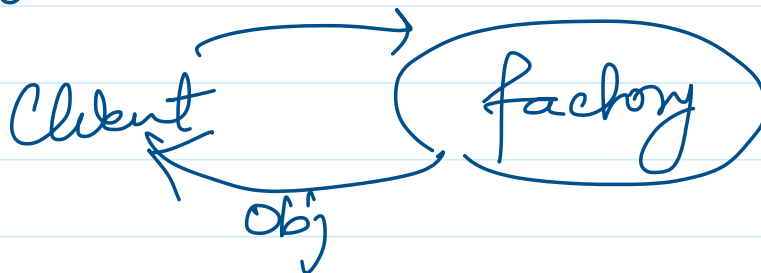
→ object is created here only

we just use them

r → talk()

w → walk()

So, now



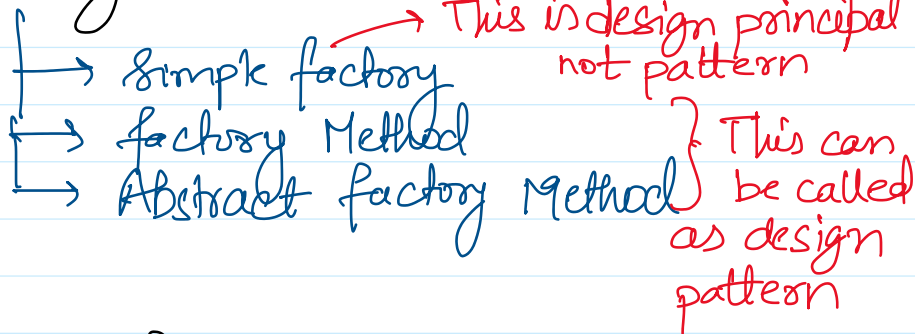
Client will ask factory for the object and factory will simply return the object.

Client doesn't know how object is created.

We simply de-coupled the Client from the Application

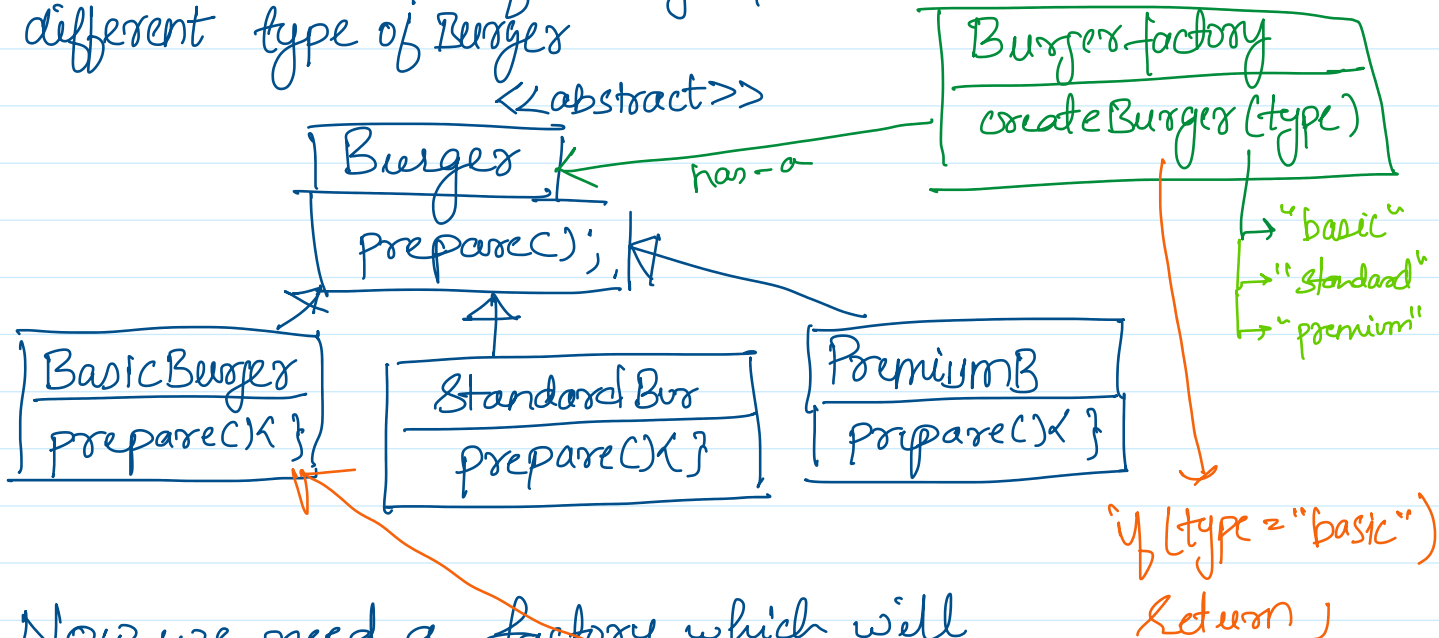
→ This is simple Goal in every design pattern, SOLID principal, or OOPS.

Factory



Simple factory

Let's take example of Burger, which has different type of Burger

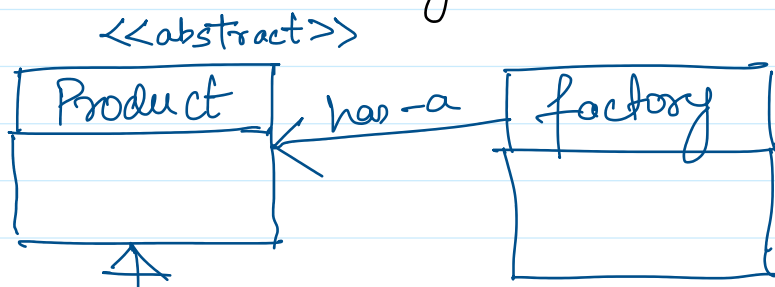


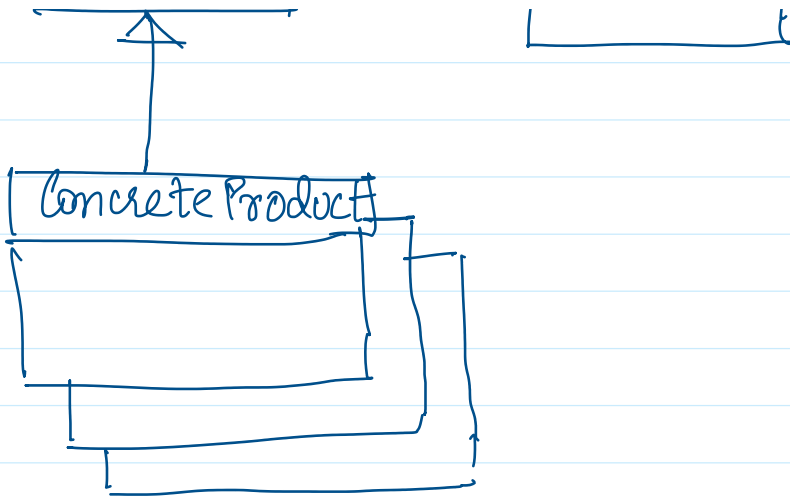
Now we need a factory which will give us ^{object of} a burger we need

This is simple factory

Code → see Notes...

Standard UML Diagram





Defination of Simple factory

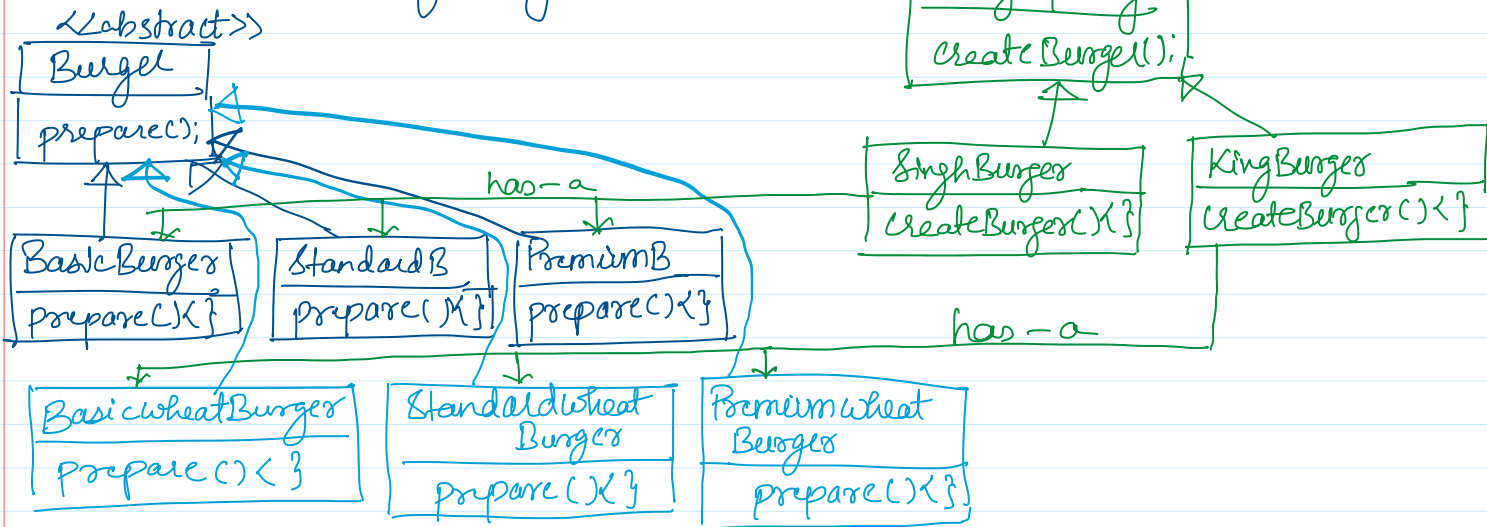
A factory class that decides which concrete class to instantiate

In Simple factory we have only one factory

In factory method we can have multiple factory

Example:-

Suppose we have 2 factories - "Singh Burger" which make Basic, Standard, Premium Burger and "King Burger" which make Healthy Burger.



At Client side:-

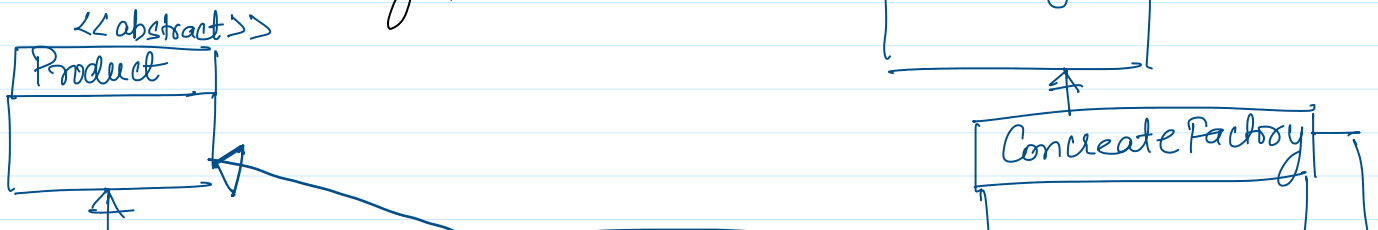
BurgerFactory * bf = new

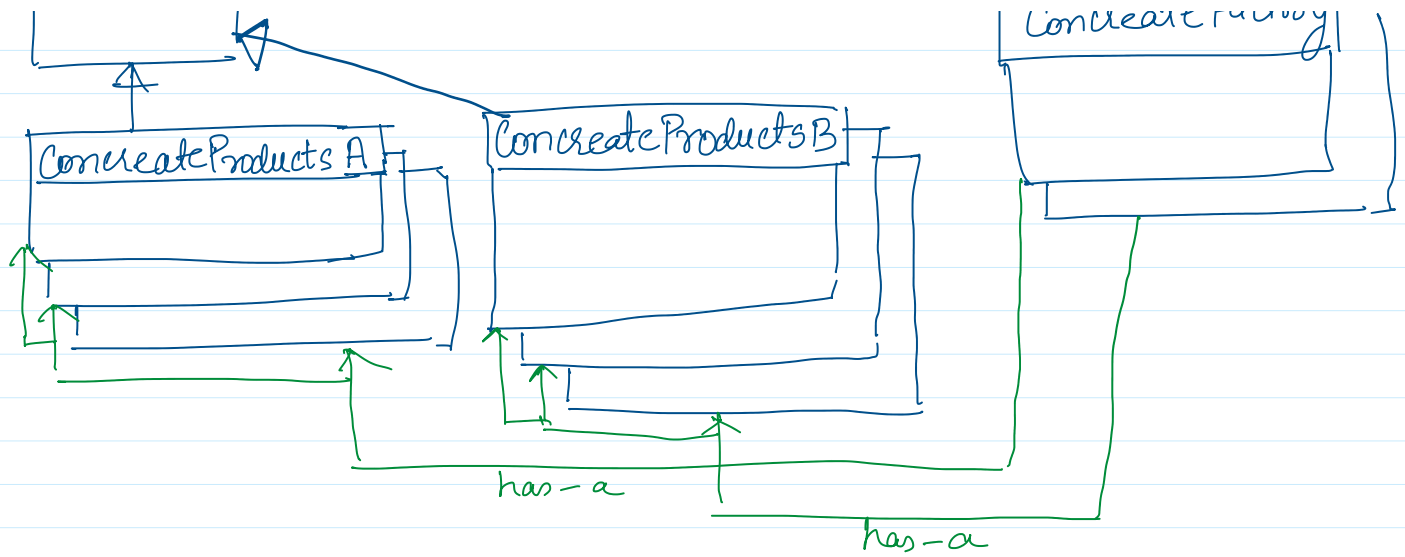
Here we will put "SinghBurger" or "KingBurger" according to the Burger we want

Code → see Notes....

So here we have put abstraction layer at factory side just like Product side.

Standard UML Diagram.



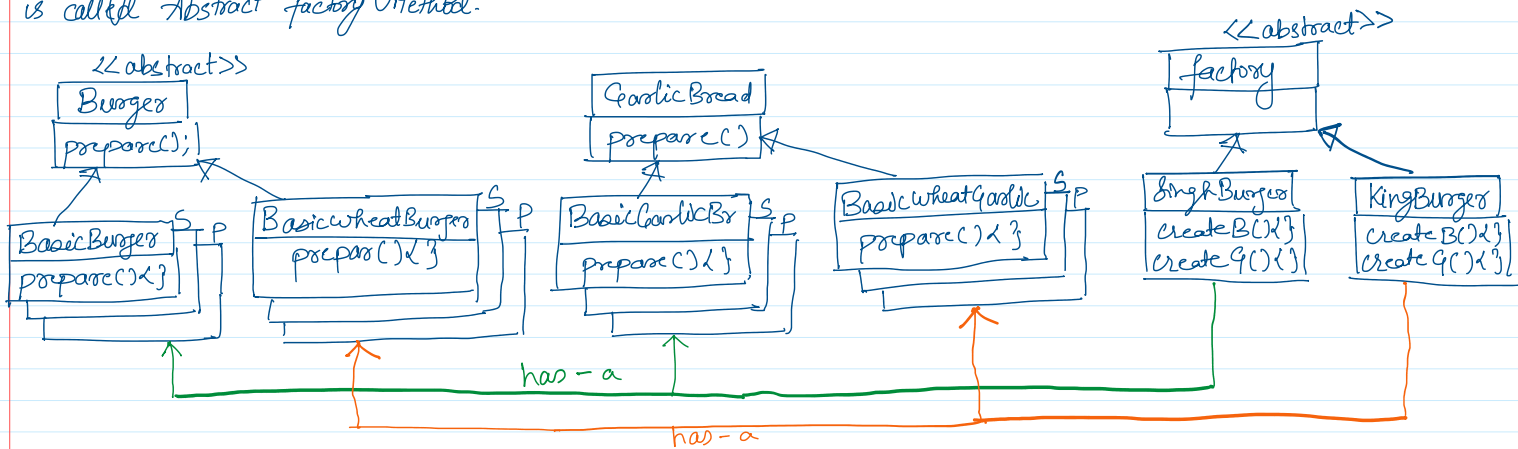


Definition of factory Method

Defines an interface for creating objects but allows subclasses to decide which class to instantiate.

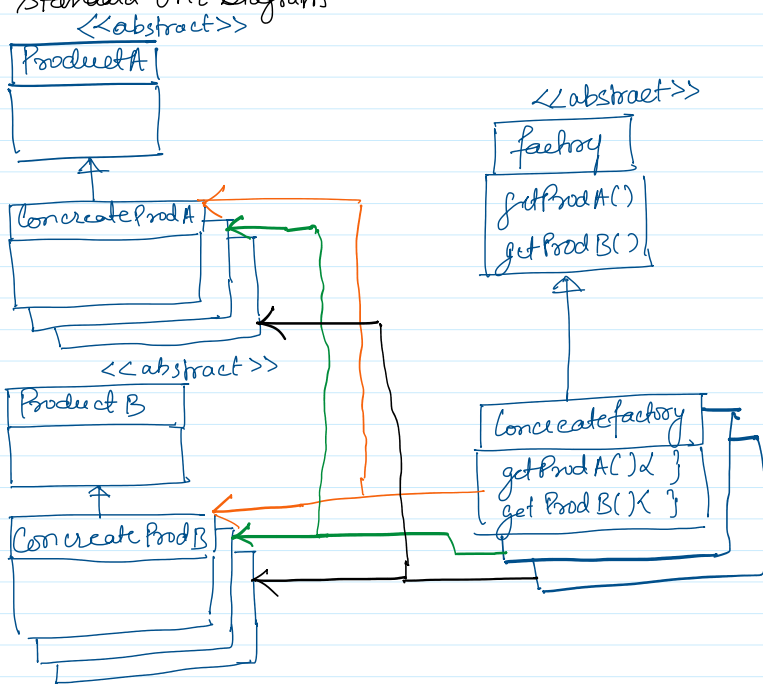
Now Suppose those "Single Burgers" & "King Burgers" starts making Garlic Bread also.

So, factory which are responsible for making more than one type of product is called Abstract factory Method.



Code → see Notes...

Standard UML Diagram



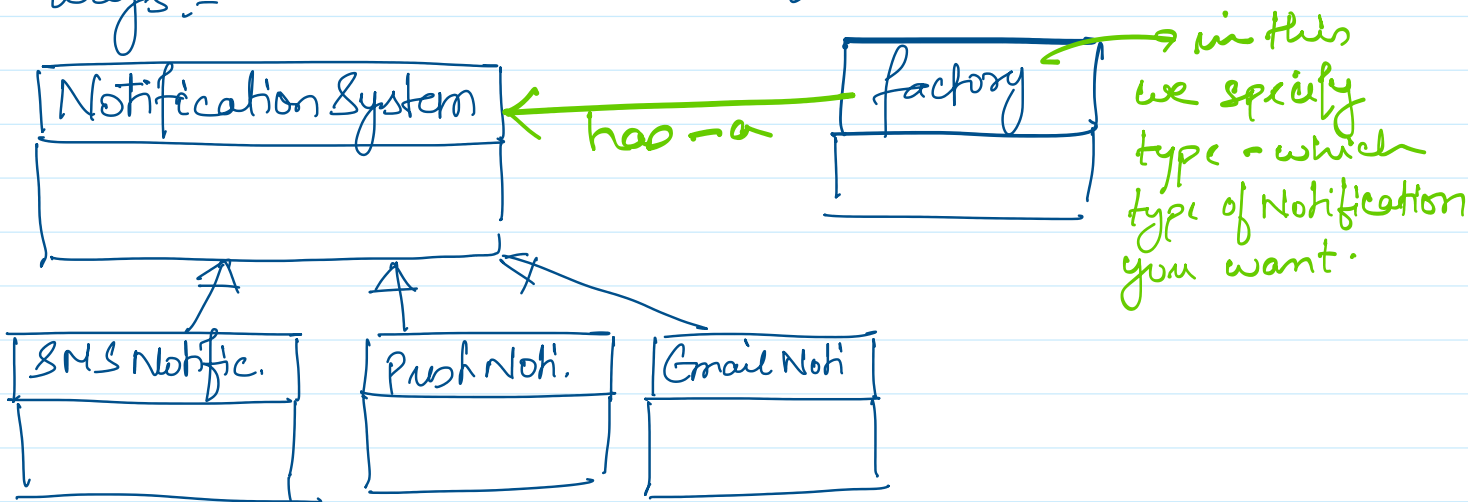
Definition of Abstract factory Method

Provides an interface for creating families of related objects without specifying those concrete classes.

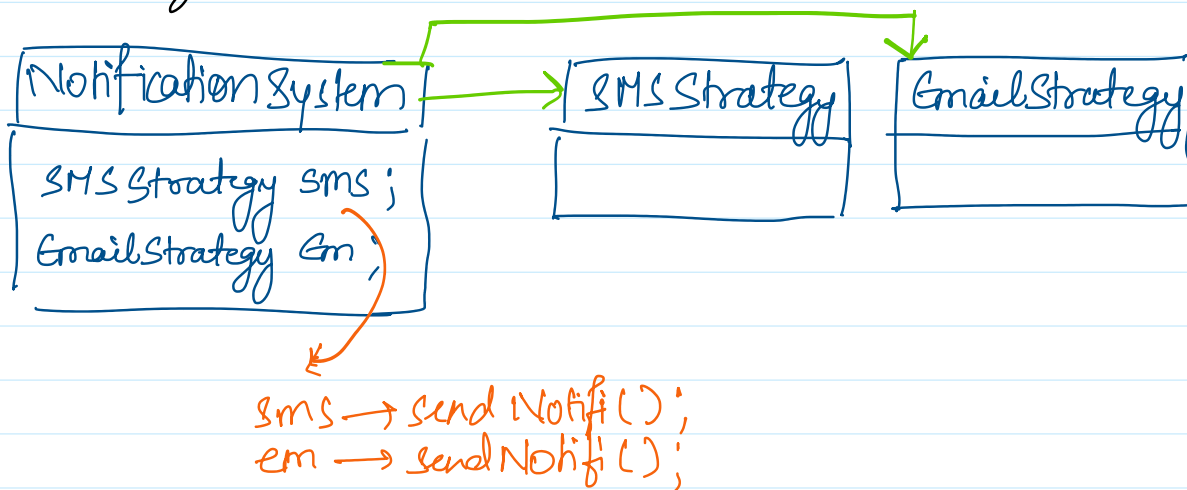
Application of factory Design Pattern

1. Notification System.

Notification can be sent in different ways :-



Here we could have also used Strategy design Pattern.



To solve same problem we can use multiple Design Pattern.
You have to decide which to use.

Strategy V/s factory

→ If you want to vary algorithms at run time — use Strategy.

~ If you want to vary algorithms at run time — use strategy.

→ If you want separate object creation logic → use factory.